

Square Root Algorithm

Square root

The square root of a number x is the number r such that $r * r = x$

- It's a simple concept mathematically, and easy for us to calculate by hand for small perfect squares (like 4, 9, 16, 25...)

In mathematics, the way this is often found is through algebra

- We rewrite the equation $r^2 = x$ as $r^2 - x = 0$
- Then we solve for r using algebraic manipulation, such as $r = x^{1/2}$

Square Root

Computing a square root exactly is actually *expensive*

- Under the hood, `x ** 0.5` or `math.sqrt(x)` still has to run some kind of iterative algorithm
- For irrational answers, we can never get a perfectly *exact* value anyway, only an approximation to some precision

So the real question becomes: how much computation are we willing to spend for how much accuracy?

As programmers, we have several strategies available to us, each with different costs

- **Guess and check (brute force)**: try every possible answer one at a time, simple, but very slow
- **Approximation**: narrow in on an answer using increments and a tolerance (epsilon), faster, still simple
- **Bisection search**: cut the search space in half at every step, much faster for ordered search spaces
- **Built-in library functions**: `math.sqrt()`, fast and convenient, but hides the algorithm from us

We'll build up from the slowest, simplest approach to the faster ones, so we understand *why* the fast ones work

Guess and check

This is also called an *exhaustive enumeration algorithm*

- Applies to a problem where
 - You are able to **guess** a value for a solution, and
 - You are able to **check** if that guess is correct
- And you can keep guessing until you either
 - Find the correct solution
 - Run out of guesses

Guess and check square root

Given an `int` called `x`,

we want to check if there exists an `int` that is the square root of `x`

1. Start with a guess, and check if it's the correct answer

Guess and check square root

2. Be *systematic*, start with a guess, then `guess + 1` , then `guess + 2` , etc

And if `x` is a perfect square, then you will eventually find the answer

Guess and check square root

And if x doesn't have a perfect square root, then we'll need to know when to stop guessing

Here we can use algebra,

if guess^2 is bigger than x , then we know that the guess is too big, and we can stop guessing

Guess and check square root

Code

Guess and check square root

```
1  guess = 0
2  x = int(input("Enter a number: "))
3  while guess**2 < x:
4      guess += 1
5
6  if guess**2 == x:
7      print("The square root of", x, "is", guess)
8  else:
9      print(x, "does not have a perfect square root")
```

Guess and check square root

- Does this work for any integer value of x ?
- What happens if x is negative?

Fixing those issues

code

Fixing those issue

```
1  guess = 0
2  is_negative = False
3
4  x = int(input("Enter a number: "))
5
6  if x < 0:
7      is_negative = True
8
9  while guess**2 < x:
10     guess += 1
11
12  if guess**2 == x:
13     print("The square root of", x, "is", guess)
14  else:
15     print(x, "does not have a perfect square root")
16
17     if is_negative:
18         print("This program does not support negative numbers")
```

Approximation

Finding the square root

Previously, we made a guess and check algorithm to find the square root of a number

- but it **only** worked for perfect squares
- what if we wanted to find the square root of any positive integer

But remember that computers do not have infinite precision

- floating point numbers are only ever an *approximation* of the real number they represent

this means we can never expect to find an answer where `guess**2 == x` **exactly**

instead, we need to accept an answer that's *close enough*

Approximation

An extension of guess and check, where you're **still** *enumerating*, but with *approximates*

1. Find an answer that's good enough
2. Run the Algorithm
 - start with a guess that we know is too small
 - increment by a small value
 - check if our guess is close to the correct answer
 - continue until we get an answer that's close enough

Implementaiton

code

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

Let's try finding the square root of

- 36
- 24
- 2
- 12345
- 54321

Will this loop always terminate?

Implementation

Let's try to debug

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11     if num_guesses % 100000 == 0:
12         print("current guess =", guess)
13         print("current guess^2 =", guess**2)
14         print("distance from x =", abs(guess**2 - x))
15     if num_guesses % 1000000 == 0:
16         input("continue?")
17
18 print("num_guesses =", num_guesses)
19 print(guess, "is close to square root of", x)
```

Implementation

Let's try to debug

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11     if num_guesses % 100000 == 0:
12         print("current guess =", guess)
13         print("current guess^2 =", guess**2)
14         print("distance from x =", abs(guess**2 - x))
15     if num_guesses % 1000000 == 0:
16         input("continue?")
17
18 print("num_guesses =", num_guesses)
19 print(guess, "is close to square root of", x)
```

Implementation

Let's try to debug

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11     if num_guesses % 100000 == 0:
12         print("current guess =", guess)
13         print("current guess^2 =", guess**2)
14         print("distance from x =", abs(guess**2 - x))
15     if num_guesses % 1000000 == 0:
16         input("continue?")
17
18 print("num_guesses =", num_guesses)
19 print(guess, "is close to square root of", x)
```

We overshoot

Diagram

The fix

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon and guess**2 ≤ x:
8      guess += increment
9      num_guesses += 1
10
11  print("num_guesses =", num_guesses)
12  if abs(guess**2 - x) ≥ epsilon:
13      print("Failed to find the square root of", x)
14  else:
15      print(guess, "is close to square root of", x)
```

The fix

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon and guess**2 ≤ x:
8      guess += increment
9      num_guesses += 1
10
11  print("num_guesses =", num_guesses)
12  if abs(guess**2 - x) ≥ epsilon:
13      print("Failed to find the square root of", x)
14  else:
15      print(guess, "is close to square root of", x)
```

The fix

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon and guess**2 ≤ x:
8      guess += increment
9      num_guesses += 1
10
11  print("num_guesses =", num_guesses)
12  if abs(guess**2 - x) ≥ epsilon:
13      print("Failed to find the square root of", x)
14  else:
15      print(guess, "is close to square root of", x)
```

The fix

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon and guess**2 ≤ x:
8      guess += increment
9      num_guesses += 1
10
11  print("num_guesses =", num_guesses)
12  if abs(guess**2 - x) ≥ epsilon:
13      print("Failed to find the square root of", x)
14  else:
15      print(guess, "is close to square root of", x)
```

Now it stops if it overshoots and reports an error

But what if we don't want to fail? What can we do

Bisection Search

Scenario

- What if I attached 100php to a page of a book
- If you can guess which page the money is on, you get to keep the money
- If you fail, you get a 5.0

Scenario

- What if I attached 100php to a page of a book
- If you can guess which page the money is on, you get to keep the money
- If you fail, you get a 5.0

What if I told you whether you were correct, too low, or too high after each guess?

Bisection Search

- Applies to problems with an inherent order to the range of possible answers
 - Examples: integers, floating point numbers, words in a dictionary, grades
- Assume we know that the answer lies in between some interval
 - In our square root, what was the start of our interval?
 - What was the end of our interval?

Bisection Search

Steps:

1. After establishing the interval
 2. Get the midpoint of the interval
 3. Check if the midpoint is close enough to the answer
 4. Check whether the midpoint is too high or too low
 5. Change the interval
 6. Repeat step 2
- This cuts the set of things to check in half at each stage
 - where exhaustive search (guess and check) reduces them from N to $N - 1$
 - bisection search reduces them from N to $N/2$

Log growth

- Cutting the number of things to check in half at each step is usually called logarithmic growth

Compared to linear growth which is how the guess and check algorithm works

Bisection search for square root

Assume we know that the answer lies between 0 and x

- Rather than exhaustively trying things starting at 0, suppose instead we pick a number in the middle of this range

If you're lucky, that's the correct answer

Bisection search for square root

if it's not close enough, check if it's too big or too small

if `guess ** 2 > x`, then we know `guess` is too big, so now search

Bisection search for square root

and if, for example, this new `guess` is such that `guess ** 2 < x`, then we know that `guess` is too small

at each stage, reduce the range of values to search by half

Bisection search takes advantage of the properties of the problem

1. where the search space is ordered
2. we can tell the guess is too high or too low

Slow square root using approximation

```
1  x = 54321 # still here
2  epsilon = 0.01
3  num_guesses = 0
4  # code here
5  # code here
6  # code here
7  while abs(guess**2 - x) ≥ epsilon:
8      # code here
9      # code here
10     # code here
11
12
13     num_guesses += 1
14
15 print('num_guesses =', num_guesses)
16 print(guess, 'is close to the square root of', x)
```

Slow square root using approximation

```
1  x = 54321
2  epsilon = 0.01
3  num_guesses = 0
4  low = 0
5  high = x
6  guess = (high + low)/2.0
7  while abs(guess**2 - x) ≥ epsilon:
8      if guess ** 2 < x:
9          low = guess
10     else:
11         high = guess
12         guess = (high + low)/2.0
13         num_guesses += 1
14
15 print('num_guesses =', num_guesses)
16 print(guess, 'is close to the square root of', x)
```

Summary

- Computing a square root exactly is expensive, so we approximate
- **Guess and check**: enumerate integers one at a time, simple, but only works on perfect squares
- **Approximation**: enumerate using increments and an epsilon, works on any positive number, but can overshoot and is slow
- **Bisection search**: cut the search space in half each step, much faster, but needs an ordered search space with a way to tell "too high" from "too low"
- Floating point numbers are never exact, so never compare them with `=`
- Always be careful that a looping condition can't be jumped over (overshooting)
- Linear growth (guess and check) vs. logarithmic growth (bisection search): fewer steps needed as the problem size increases